

R-Kurs WiSe 25/26: Aufgabenblatt 8

05.12.2025

Contents

Aufgabe 1: Polygone in S3 (5x4 = 20 Punkte)	2
Aufgabe 2: Sportspiel in S4 (20 Punkte)	5
Aufgabe 3: Turtle light in R6 (20 Punkte)	8
Aufgabe 4: Biologische Sequenzen in S7 (20 Punkte)	10

Erstellen Sie pro Aufgabe eine R-Code-Datei und benennen Sie diese nach dem Schema P-.R also hier P08-1.R, P08-2.R usw. Schreiben Sie den Code zur Lösung einer Aufgabe in die jeweilige Datei. Bei Aufgaben, in denen Sie auch Fragen beantworten sollen, schreiben Sie die Antwort als Kommentar mit in die R-Datei.

Sie dürfen in Gruppen von *bis zu vier* Personen abgeben.

Bei Abgabe von mehr als einer Person gibt nur eine dieser Personen ab. Dabei müssen **in jeder abgegebenen Datei** in der ersten Zeile als Kommentar die Namen **aller Gruppenmitglieder** stehen also etwa

```
# Alice Sender, Bob Receiver, Eve Interceptor  
  
1+1  
# ...
```

Fehlt dieser Eintrag, so wird die Aufgabe nur für die einreichende Person gewertet.

Die Abgabe der **einzelnen Dateien (kein Archiv wie .zip oder .tar)** erfolgt über Moodle im Element namens A08. Die Abgabe muss bis spätestens Freitag, 12.12.2025, 11:00 erfolgen.

Die Aufgaben auf dem aktuellen Blatt sind 80 Punkte wert. Jede Gruppe sollte sich drei der vier Aufgaben aussuchen und dazu Lösungen einreichen. Damit sind dann wie üblich maximal 60 Punkte zu erreichen.

Aufgabe 1: Polygone in S3 (5x4 = 20 Punkte)

Unter einem Objekt der S3-Klasse `Polygon` wollen wir eine Liste mit den Einträgen `sides` und `angles`, die jeweils numerische Vektoren der gleichen Länge (Anzahl der Seiten des Polygons, ≥ 3) sind. Dabei sind `sides` die (positiven) Seitenlängen und `angles` die Winkel (zwischen 0 und 2π , also in Bogenmaß). Dabei ist `angles[i]` der Winkel zwischen `sides[i]` und `sides[i+1]` bzw. im Fall $i = \text{length}(\text{sides})$ zwischen `sides[length(sides)]` und `sides[1]`.

Als oft genutzte Unterklassen wollen wir `Quadrilateral` (Viereck) und `Triangle` (Dreieck) betrachten. Zudem gibt es zu `Quadrilateral` noch die weitere Unterklasse `Rectangle`.

1. Auf Instanzen dieser Klasse wollen wir die Funktion `plot_polygon` anwenden, die ein gegebenes Polygon zeichnet. Diese Funktion soll auch aufgerufen werden, wenn `plot(poly1)` aufgerufen wird, sofern es sich bei `poly1` um ein Objekt der Klasse `Polygon` handelt. `plot_polygon` beruht darauf, dass wir von einer gegebenen `polygon`-Instanz die Eckpunkte mittels der Funktion `get_corners` erhalten können:

```
poly1 <- structure(
  list(sides = c(2, 2, 2, 2), angles = c(pi/3, 2*pi/3, pi/3, 2*pi/3)),
  class = c("Quadrilateral", "Polygon"))
poly2 <- structure(
  list(sides = c(3, 4, 3, 4), angles = rep(pi/2, 4)),
  class = c("Rectangle", "Quadrilateral", "Polygon"))
poly3 <- structure(
  list(sides = c(2, 2, 2), angles = rep(pi/3, 3)),
  class = c("Triangle", "Polygon"))
poly4 <- structure(
  list(sides = rep(1, 5), angles = rep(3/5*pi, 5)),
  class = c("Polygon"))

get_corners <- function(x) {
  n <- length(x$sides)
  if (n < 3) return(invisible(NULL))
  out <- matrix(ncol = 2, nrow = length(x$sides))
  direction <- c(1, 0)
  point <- c(0, 0)
  out[1, ] <- point
  for (i in 1:(n-1)) {
    point <- point + direction * x$sides[i]
    out[i+1, ] <- point
    a <- pi - x$angles[i]
    direction <- matrix(c(cos(a), sin(a), -sin(a), cos(a)), ncol=2) %*% direction
  }
  out
}

plot_polygon <- function(x) {
  corners <- get_corners(x)
  plot(range(corners[,1]), range(corners[,2]), type = 'n', axes=FALSE, ann=FALSE, asp=1)
  polygon(corners[,1], corners[,2], lwd = 3)
}

plot(poly1)
plot(poly2)
```

```
plot(poly3)
plot(poly4)
```

Wir erhalten als Ausgabe folgende Bilder:

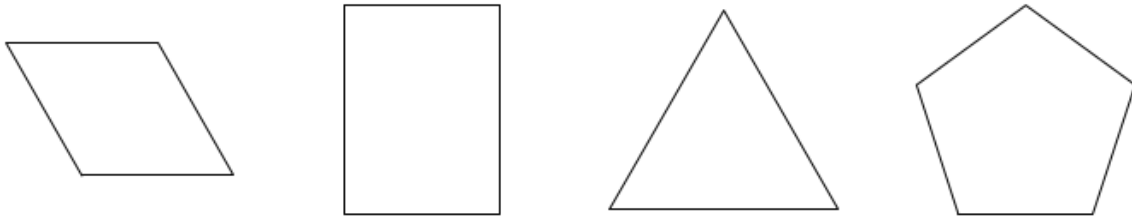


Figure 1: Einige Polygone

- Als Konstruktor im S3-Stil schreiben wir die Funktion **Rectangle**, die zwei Variablen **w** und **h** entgegennimmt und ein Objekt der Klasse **Rectangle** der Breite **w** und Höhe **h** erstellt. Ferner soll eine Funktion **Triangle** wahlweise einen numerischen Vektor **s** der Länge 3 oder einen numerischen Vektor **s** der Länge 2 und eine weitere Zahl **a** entgegennehmen und ein **Triangle** generieren, das entweder die von **s** gegebenen Seitenlängen (**sides** = **s**) oder **sides[i]** = **s[i]** mit $i = 1, 2$ und **a[1]** = **a**.

Hinweis: Die relevanten Formeln für elementargeometrische Körper finden Sie in jedem Tafelwerk oder auf geeigneten Internetseiten, z.B. <https://de.wikipedia.org/wiki/Dreieck#Formeln>

- Gerade bei Dreiecken kann es durchaus vorkommen, dass einem bei den Seiten und Winkeln mal etwas durcheinander gerät. Da kommt uns die nächste Funktion **validate_triangle** gerade recht, die ein einzelnes Argument **x** entgegennimmt und prüft, ob es sich dabei um ein **Triangle** mit richtigen Seiten- und Winkelverhältnissen handelt:

```
triangle1 <- triangle(s = c(3,4,5))
validate_triangle(triangle1) # keine Ausgabe
triangle1$angles[1:2] <- triangle1$angles[2:1]
validate_triangle(triangle1)
## Error in validate_triangle(triangle1): Sides and angles do not fit.
```

- Weiter suchen wir eine generische Funktion **circumf**, die ein Objekt der Klasse **Polygon** schluckt und dessen Umfang ausspuckt, etwa so:

```
circumf(triangle(c(3,4), pi/2))
## [1] 12
circumf(rectangle(3.1,2))
## [1] 10.2
```

- Für Objekte der Klassen **Triangle** oder **Quadrilateral** wollen wir auch den Flächeninhalt mittels einer generischen Funktion **area** berechnen können:

```
area(triangle(c(3,4,5))
## [1] 6
area(rectangle(3.1, 4))
## [1] 6.2
area(poly1)
## [1] 3.464102
area(poly2)
## [1] 12
area(poly3)
## [1] 1.732051
area(poly4)
## Error in area.polygon(poly4): area formula not implemented for polygons with more than 4 sides
area(c(3,5))
## Error in area.default(c(3,5)): no area formula implemented for objects of this 'class'
```

Aufgabe 2: Sportspiel in S4 (20 Punkte)

Wir schreiben eine Klasse `Game`, deren Instanzen Spiele verschiedener Sportarten abbilden sollen. Zentral soll eine Simulation des Spiels sein, an deren Ende ein Ergebnis steht. Die Spiele sollen wahlweise im Satzmodus (z.B. Volleyball, Tischtennis) oder auf Zeit (z.B. Fußball, Handball) ablaufen. Wir brauchen also zwei Kind-Klassen `SetGame` und `TimeGame`.

Bei Spielen im Satzmodus wird vorgegeben,

- (i) wie viele Sätze gewonnen werden müssen, bis das Spiel gewonnen wird (es kann ggf. auch nur einer sein) und
- (ii) wie viele Punkte (Ballwechsel, Legs, ...) zum Satzgewinn nötig sind, und ob zwei Punkte Vorsprung nötig sind.

Bei Zeitspielen geben wir an, wie viele Punkte (bzw. Tore) wir pro Spiel (im Durchschnitt) erwarten. Das wird für verschiedene Spiele sehr unterschiedlich sein.

Die Simulation soll bei Spielen im Satzmodus so ablaufen, dass wir jeden Satz Punkt für Punkt simulieren, bis eine Seite den Satz gewonnen hat. Das geschieht Satz für Satz, bis eine Seite genug Sätze gewonnen hat, um das Spiel zu gewinnen.

Bei Spielen auf Zeit geben wir die Anzahl der zu spielenden Minuten vor. Für jede Minute ist dann die Zahl der in dieser Minute fallenden Tore Poisson-verteilt mit der Rate

$$\lambda = \frac{1}{T} \mathbb{E}(\#\{\text{Tore pro Spiel}\}),$$

wobei T die Spieldauer in Minuten ist. Dann ziehen wir zu jeder Spielminute die Anzahl der Tore und verteilen diese (gleich wahrscheinlich) auf die Teams.

Jedes Spiel braucht folgende Angaben:

1. Zwei Parteien (Teams oder Spieler*innen),
2. einen Spielverlauf (anfangs per default leer) und
3. einen Status ("`open`", "`in progress`" oder "`final`"), wobei "`open`" der Status ist, den das Spiel per default bekommt, wenn es angelegt wird.

Ein Tischtennisspiel `tt1` zwischen "A. B." und "C. D." erstellen wir also als Instanz der Klasse `TTGame` mit

```
tt1 <- new(TTGame, parties = c("A. B.", "C. D."))
```

Außerdem brauchen wir die Methode `play_game` (entgegen der empfohlenen Notation, generische S4-Methoden im `lowerCamelCase` zu schreiben - diese behalten wir uns für Variablennamen), die per Simulation das Ergebnis erzeugt und den Status des Spiels (der Instanz) während der Simulation auf "`in progress`" und nach erfolgter Simulation auf "`final`" setzt. Ist das Spiel nicht mehr im Zustand `open`, so soll dieser Umstand mitgeteilt und das Spiel nicht erneut simuliert werden (Wiederholungen von Spielen sind nicht vorgesehen).

Frage: Warum brauchen wir den Status "`in progress`", statt einfach von "`open`" zu "`final`" zu wechseln?

Das Ergebnis soll den Spielverlauf wiedergeben. Das bedeutet, dass man einem beendeten Spiel ansehen soll, in welcher Reihenfolge die Punkte erzielt wurden. Mit `show_game_course` soll dieser Spielverlauf ausgegeben werden. Hat etwa Spielerin "A. B." das Tischtennisspiel `tt1` in 3:1 Sätzen gegen "C. D." verloren, könnte das Ergebnis so aussehen:

```

> tt1 <- play_game(tt1) #or just: play_game(tt1), but written in a way that alters tt1
> show_game_course(tt1)
## Parties:
##      A      B
## "A. B." "C. D."
##
## Set 1  9:11
## B A B A B B A B A A A A B B A A B B B B
## Set 2  5:11
## B B B A B B A B B B A A A B B B
## Set 3  11:8
## A A B B A B A A B B A B B B A A A A A
## Set 4  2:11
## B B B B B B A A B B B B

```

Weiter soll eine Methode `show` das Ergebnis in der typischen Form angeben.

```

> show(tt1)
## A game of table tennis: final
## A. B. vs. C. D.
## 1:3 (9:11 5:11 11:8 2:11 )

```

Bevor das Spiel gespielt wurde, hätte die Ausgabe hingegen so ausgesehen:

```

> show(tt1)
## A game of table tennis: open
## A. B. vs. C. D.
## No results yet.

```

Erstellen Sie also die S4-Klasse `Game` mit ihren Kindklassen `SetGame` mit Kindklassen `TTGame` (Tischtennis) und `VBGame` (Volleyball) und `TimeGame` mit Kindklassen `FBGame` (Fußball) und `HBGame` (Handball) haben.

```

setClass("Game",
  slots = c(
    parties = #ToDo
  ),
  prototype = #ToDo
)

```

Schreiben Sie auch die oben beschriebenen Methoden `play_game`, `show` und `show_game_course`, die ein Spiel (nach den für die Sportart vorgegebenen Regeln) simulieren und das Spielergebnis (sofern schon vorhanden) bzw. den Spielverlauf ausgeben. Unter anderem sollte etwas in dieser Form stehen:

```

setGeneric("play_game", function(x) standardGeneric("play_game"))
setMethod("play_game", "Game", function(x) {
  #ToDo;
  #hier sollte mindestens der Status des Spiels `x` auf `"in progress"` gesetzt werden
})
setMethod("play_game", "SetGame", function(x) {
  #ToDo;
  #implementiere hier die eigentliche Simulation eines Satzspiels
})

```

```

setMethod("play_game", "TTGame", function(x) {
  # rufe Methode play_game für "SetGame" mit den für Tischtennis gegebenen Parametern
  # auf und verpacke das erhaltene Ergebnis in das Ergebnis
})
# ....
setMethod("show", "Game", function(x) {
  #ToDo: Erstelle hier
})

```

Hinweis: Zur Implementierung von `play_game` kann im Fall eines Satz-Spiels eine Hilfsfunktion `play_set` hilfreich sein.

Folgende Parameter sollen zur Simulation der Spiele vorgegeben sein:

Beim Tischtennis werden 3 Gewinnsätze gespielt, zum Satzgewinn sind 11 Punkte nötig. Stehen beide Parteien in einem Satz bei 10 Punkten, so wird weiter gespielt, bis eine Partei zwei Punkte Vorsprung hat. Beim Volleyball werden drei Gewinnsätze bis 25 Punkte gespielt, wobei wieder zwei Punkte Vorsprung nötig sind, um einen Satz zu gewinnen. Haben beide Teams je zwei Sätze gewonnen, so wird ein Tiebreak bis 15 Punkte (wieder: mind. 2 Punkte Vorsprung) gespielt.

Hinweis: Zur Simulation eines Satzes kann eine `while`-Schleife hilfreich sein. Eine solche darf hier benutzt werden.

Beim Fußball werden nach unserer Annahme pro Spiel im Schnitt 3.7 Tore in 90 Minuten erzielt, beim Handball 48 Tore in 60 Minuten. Wir ignorieren die zunehmend ausufernde Nachspielzeit im Fußball und pfeifen nach 90 Minuten ab. Auch Verlängerungen oder ähnliches werden (wie im Ligabetrieb) nicht berücksichtigt.

Aufgabe 3: Turtle light in R6 (20 Punkte)

Wir erstellen eine R6-Klasse `GridPath`, die auf einem Gitter Pfade zeichnen kann, indem ihre Instanzen auf Bewegungsbefehle reagieren und durchlaufene Punkte des Gitters (sowie die Kanten zwischen diesen Punkten) schwarz gefärbt werden. Objekte dieser Klasse haben ein Feld `path`, in dem in Form einer $n \times 2$ -Matrix die $n \geq 1$ (x, y) -Koordinaten der bereits durchlaufenen Gitterpunkte gespeichert sind, sowie einen `character`-Vektor `direction` der Länge 1, dessen Eintrag ("R", "L", "U" oder "D") für "right", "left", "up" und "down" die aktuelle Laufrichtung vorgibt.

Als Methoden brauchen wir `move`, welche als Argument eine natürliche Zahl `steps` entgegennimmt, die besagt, um wie viele Gitterpunkte wir in die Blickrichtung gehen sollen. Weiter gibt es noch `rotate_left` und `rotate_right`, die unsere Blickrichtung `direction` um 90 Grad in die angegebene Richtung rotiert, ohne dass wir uns von der Stelle bewegen:

```
pfad1 <- GridPath$new()
pfad1$direction <- "U"
pfad1$rotate_right()
pfad1$direction
## [1] "R"
pfad1$rotate_right()
pfad1$direction
## [1] "D"
pfad1$rotate_left()$rotate_left() # method chaining; Wir könnten alles aneinander hängen
pfad1$direction
## [1] "U"

pfad1$path <- matrix(c(0, 0), nrow=1)
pfad1$direction <- "U"
pfad1$move(3)
pfad1$path
##           [,1] [,2]
## [1,]      0   0
## [2,]      0   1
## [3,]      0   2
## [4,]      0   3
pfad1$rotate_right()
pfad1$move(2)
pfad1$path
##           [,1] [,2]
## [1,]      0   0
## [2,]      0   1
## [3,]      0   2
## [4,]      0   3
## [5,]      1   3
## [6,]      2   3
```

Hinweis: Nutzen Sie eine Liste der Form `list(U = c(0,1), R = c(1,0), L = c(-1,0), D = c(0,-1))` zur Umsetzung der Bewegung bei gegebener Blickrichtung.

1. Erstellen Sie die Klasse mit der angegebenen Methoden nach diesen Angaben.
2. Wir fügen der Klasse `GridPath` noch eine Methode `initialize` hinzu, die als fakultative Argumente `direction` (default: "U") und den Startpunkt (default: `start = c(0, 0)`) annimmt und einen `GridPath` mit aktueller Blickrichtung `direction` und aktuellem Pfad `matrix(start, nrow=1)` erstellt. Diese wird als Konstruktor genutzt und wird bei `$new` aufgerufen:

```

pfad2 <- GridPath$new("W")    # ruft initialize("W") auf
pfad2$direction
## [1] "W"
pfad2$path
##      [,1] [,2]
## [1,]    0    0

```

3. Wenn wir die Pfade auf unserem Gitter konstruiert haben, möchten wir sie auch anzeigen. Dazu bekommt unsere Klasse `GridPath` eine weitere Methode `print()`. Diese soll den zurück gelegten Pfad (die besuchten Punkte und die Verbindungsgeraden dazwischen; Linientyp "b" bei `plot`) sowie an der aktuellen Position die aktuelle Blickrichtung als Pfeil (nutzen Sie etwa `arrow`) darstellen. Diese Methode soll durch Eingabe eines Pfades, also einfach durch Eingabe von `pfad1` aufgerufen werden. Genauer ruft `pfad1` automatisch `print(pfad1)` auf, dieses wiederum `print.R6(pfad1)`. Im Endeffekt verweisen all diese Aufrufe auf `pfad1$print()`.

Zusatz (ohne Wertung). *Wir sind nun nur noch wenige Schritte entfernt von einem Zufallsweg (random walk). Stellen Sie sich nur einen GridPath vor, der bei $c(0,0)$ (also im Ursprung) mit einer zufällig gewählten Richtung beginnt und immer einen Schritt (`move(1)`) geht, danach zufällig nach links oder rechts dreht (`rotate_left()` oder `rotate_right()`) oder die Richtung beibehält (`do nothing`) und dann wieder einen Schritt geht. Sie können nun zum Beispiel (experimentell mittels vieler solcher Pfade) untersuchen, wie die Anzahl der Schritte verteilt ist, bis zum ersten Mal ein Punkt erneut besucht wird. Sie können nun mit sehr langen Pfaden (und der durch die Methode `print` automatisch durchgeführten Skalierung) einen Eindruck davon gewinnen, wie typische Pfade einer zweidimensionalen Brownschen Bewegung aussehen. Machen Sie Ihren Zufallspfad gern auch selbstvermeidend (self avoiding random walk): Sollte die zufällig gewählte neue Richtung dazu führen, dass der nächste Schritt zu einem bereits besuchten Punkt führt, wählen Sie zufällig aus den verbleibenden möglichen Richtungen. Wie viele Schritte können Sie nun gehen, bis kein Schritt mehr ohne Selbstschneiden möglich ist? Auch hier können Sie mit vielen Pfaden wieder ein Gefühl der Verteilung der maximalen Anzahl dieser durchführbaren Schritte entwickeln.*

Aufgabe 4: Biologische Sequenzen in S7 (20 Punkte)

Wir entwerfen eine S7-Klasse `BioSequence` für biologische Sequenzen und dazu drei Kindklassen `DNASequences`, `RNASequence` und `ProteinSequence`. Jede Sequence hat einen Namen und die Sequenz selbst (als String) als Felder. Zudem ist zu beachten, dass DNA-Sequenzen nur die Buchstaben A, T, C, G, RNA-Sequenzen nur die Buchstaben A, U, G, C und Proteinsequenzen nur die Buchstaben A, C, D, E, F, G, H, I, K, L, M, N, P, Q, R, S, T, V, W, Y enthalten sollen. Das Erstellen eines Objekts der Klasse `DNASequences` mit der Sequenz `AUGACG` sollte also mit einer Fehlermeldung scheitern.

Hinweis: Hierzu braucht die Klasse einen Validator, den Sie am besten gleich in die Klassendefinition schreiben. Dieser muss nicht auf weitere Fehleingaben prüfen.

Weiter brauchen wir folgende generische Methoden und dazu die jeweiligen Implementierungen:

1. `grow_seq()`, die eine Sequenz `seqIn` und einen String `addSeq` entgegen nimmt und letzteren an die Sequenz anhängt, sofern dieser für die fragliche Sequenzklasse zulässig ist. Soll etwa an eine RNA-Sequenz ein "ATG" angehängt werden, sollte das in einer Fehlermeldung (wegen des "T"s) enden.
2. `splice_seq()`, die eine Sequenz `seqIn` und eine (positive ganze) Zahl `n` entgegen nimmt und die gegebene Sequenz zurück gibt, falls die Zahl mindestens so groß wie die Sequenz lang ist. Ist `n` kleiner als die Länge der Eingabesequenz, so soll eine Liste aus zwei Sequenzen (der gleichen Klasse wie die Eingabeklasse) zurück gegeben werden, deren erste die ersten `n` Einträge von `seqIn` und die zweite den Rest enthält.
3. `translate_seq()`, die eine Sequenz `seqIn` und als String einen Klassennamen `to` entgegen nimmt und die übersetzte Version der Eingabesequenz ausgibt. Außerdem muss noch der Name der Ausgabesequenz als String `name` übergeben werden. Ist `seqIn` bereits von der durch `to` spezifizierten Klasse, so muss nichts passieren außer gegebenenfalls der Umbenennung. Andernfalls soll z.B. die DNA-Sequenz "ATG" unter `to = RNASequence` zur RNA-Sequenz "AUG" umgewandelt werden. So soll etwa eine DNA-Sequenz "ATG" in die Proteinsequenz "M" übersetzt werden (AUG kodiert für Methionin). Bei der Umwandlung einer DNA- oder RNA-Sequenz ist dabei darauf zu achten, dass die Sequenz mit einem gültigen Startkodon (TTG/UUG, ATG/AUG oder GTG/GUG) beginnen muss. Wird ein Stoppkodon erreicht, so soll die Übersetzung abgebrochen werden, auch wenn die eingegebene Kette noch nicht beendet ist. Wird das Ende der Kette erreicht, so bricht die Übersetzung einfach mit dem letzten vollständigen Kodon (Dreierblock von Basen) ab, auch wenn kein Stoppkodon gefunden wurde. Die Rückübersetzung von Proteinsequenzen in RNA- oder DNA-Sequenzen müssen Sie nicht implementieren (da wäre ohnehin einiges an Raten gefragt). Bei der Implementierung kann sich die Methode `splice_seq` als hilfreich erweisen.

Hinweis: Sie finden eine Übersetzungstabelle von RNA in Aminosäuren inklusive Start- und Stoppkodons zum Beispiel unter

https://en.wikipedia.org/wiki/DNA_and_RNA_codon_tables#Translation_table_1

Hier wieder einige Beispielein- und ausgaben:

```
library(S7)
BioSequence <- ## ... a lot ToDo

dna1 <- DNASequences(name = "dna1", sequence = "ATGACGTAGAAU")
splice_seq(dna1, 5)[[1]]@sequence
## [1] "ATGAC"
splice_seq(dna1, 5)[[2]]@sequence
## [1] "GTAGAAU"

prot1 <- translate_seq(dna1, to = "ProteinSequence", name = "prot1")
prot2 <- ProteinSequence(name = "prot2", sequence = "MT")
```

```
prot1@sequence == prot2@sequence
## [1] TRUE
dna2 <- DNASquence(name = "dna2", sequence = "AUGACG")
## Error: <DNASquence> object is invalid:
## - @sequence contains illegal characters
rna1 <- RNASquence(name = "rna1", sequence = 124231)
## Error: <BioSequence> object properties are invalid:
## - @sequence must be <character>, not <double>
```

Hinweis: Der letzte Fehler wird automatisch generiert, wenn Sie Ihre Klassen richtig definiert haben. Um den müssen Sie sich nicht kümmern.